

МИНИСТЕРСТВО ОБРАЗОВАНИЯ И НАУКИ РФ
ФГБОУ ВО Волгоградский государственный технический университет
Кафедра «ПРОГРАММНОЕ ОБЕСПЕЧЕНИЕ
АВТОМАТИЗИРОВАННЫХ СИСТЕМ»

Основы работы в распределенных системах контроля версий

Методические указания



Волгоград
2018

Рецензент

к. т. н., доц. Н.П. Садовникова

Издается по решению редакционно-издательского совета
Волгоградского государственного технического университета

Основы работы в распределенных системах контроля версий: метод. указания / сост. О. А. Сычев, Г.В. Терехов; ВолгГТУ. – Волгоград, 2018. – 17 с.

В методических указаниях изложены цели, содержание и порядок выполнения лабораторной работы «Запрос коммитов и их диапазонов» по дисциплине «Распределенный контроль версий кода», приведены основные сведения о базовых командах работы в современных системах распределенного контроля версий Git и Mercurial, рассмотрены способы указания ссылок на коммиты и их диапазоны в качестве параметров различных команд.

Методические указания предназначены для изучения дисциплин «Эволюция и сопровождение программного обеспечения» и «Распределенный контроль версий кода». Методические указания могут быть использованы студентами, обучающимися по направлению 09.03.04 «Программная инженерия», а также широким кругом лиц, интересующихся программированием.

© Волгоградский государственный
технический университет, 2018

Оглавление

<i>1. Повседневное использование распределенных систем контроля версий. .</i>	<i>4</i>
1.1. Особенности работы в распределенных системах контроля версий на примере Mercurial.....	4
1.2 Ссылки на коммиты и их диапазоны в Mercurial.....	6
1.3. Особенности работы в системе Git.....	10
1.4. Ссылки на коммиты и их диапазоны в Git.....	16

1. Повседневное использование распределенных систем контроля версий

Базовые команды централизованных систем контроля версий нередко сохраняют свой смысл в распределенных, например для систем Git и Mercurial это:

1. `commit` – зафиксировать (закоммитить) совершенные изменения (Git, Mercurial).

2. `merge` – слить вместе изменения в двух различных ветках (Git, Mercurial) – однако, в отличие от централизованных систем, ветки могут быть анонимными (созданными в результате параллельной работы программистов);

3. `update` – обновить состояние рабочего каталога к состоянию на момент определенной ревизии (только Mercurial, в Git эту функцию выполняет команда `checkout`).

Команда, применяющаяся в начале работы над проектом и получающая с сервера необходимые данные, изменила свой смысл (теперь она получает не содержимое рабочего каталога, а создает копию репозитория) и название – `clone` вместо `checkout`. Однако наибольшим изменением является введение команд для обмена данными между репозиториями: вытягивание для получения коммитов из удаленного репозитория (`pull`, `fetch`) и выталкивание для отправки коммитов в удаленный репозиторий (`push`).

1.1. Особенности работы в распределенных системах контроля версий на примере Mercurial

Первоначально с точки зрения Mercurial история кода представлялась неизменной – коммит можно отменить, вводя «противоположный», но не удалить или изменить. Это дает гарантию корректного слияния кода, если работа клонировалась из общего репозитория, но затрудняет некоторые типовые приемы работы, связанные с созданием идеализированной истории коммитов. Поэтому позже возможность редактировать коммиты была введена, но с ограничениями, для чего было введено понятие фазы коммита. Коммит может быть в одной из трех фаз: публичной (был опубликован и доступен для вытягивания другим, не может быть изменен), черновик (может быть изменен) и секретный (не показывается и не участвует в обычных командах, используется внутренне для реализации некоторых команд (см., например, очереди патчей). По умолчанию коммит создается черновиком и может быть отредактирован до выталкивания на внешний сервер, но при выталкивании становится публичным (как в том репозитории, куда выталкивают, так и в том, откуда выталкивают), т.к. выталкивание обычно происходит на внешний, доступный другим

программистам сервер. Однако, если есть потребность иметь сервер для предварительного просмотра другими программистами работы с возможностью последующего редактирования коммитов, можно в настройках принудительно выставить его непубликующим (non-publishing), в этом случае вытолкнутые коммиты сохраняют свой статус черновика. Обычно непубликующими делаются репозитории, предназначенные для предварительного просмотра работы начальником или старшим программистом и недоступные для других членов команды.

Для корректного понимания работы Mercurial и сообщений от него необходимо знать следующие понятия. Head (голова, вершина) – это любой коммит, который не имеет потомков; в данный момент репозиторий (и любая именованная ветка в нем) может иметь несколько head-коммитов, самой типичной ситуацией с несколькими head-коммитами является вытягивание изменений от других программистов, сделанных параллельно со своими. Для уменьшения количества head-коммитов используется команда merge. В свою очередь tip (наконечник) – это коммит, который в данном репозитории был сделан последним. Tip всегда один и всегда является head, но не каждый head есть tip (обратите внимание, что tip хотя и похож на HEAD в Git, но не является его аналогом: если привести рабочий каталог в состояние промежуточного (не-head) коммита, то tip останется наверху; в то время как HEAD в Git указывает именно на текущий коммит). Default – имя ветки, создаваемой в репозитории по умолчанию (см. рис. 1).

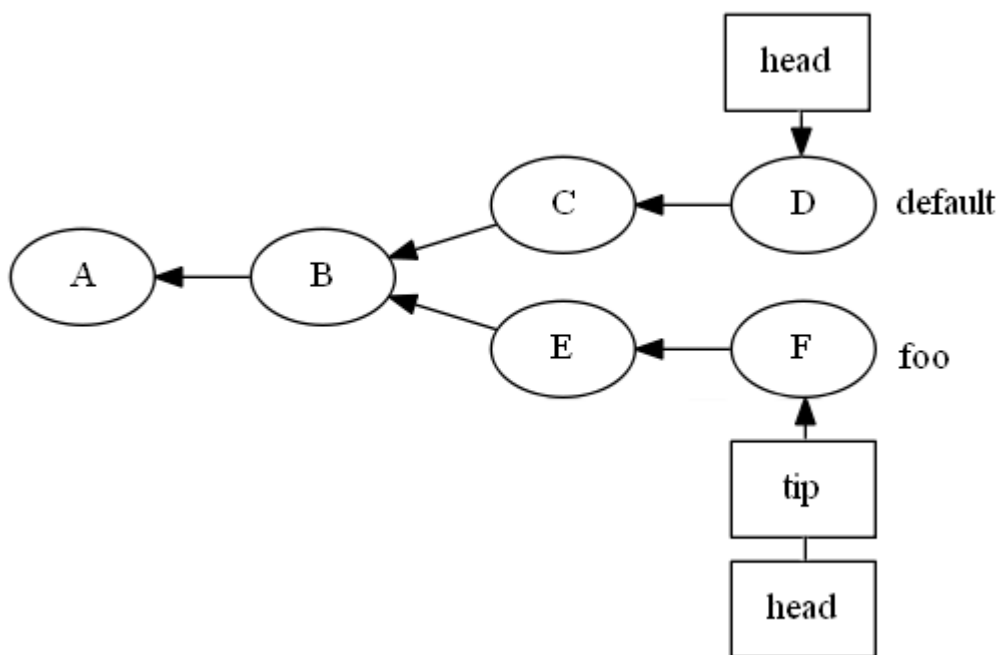


Рисунок 1. Репозиторий Mercurial с двумя ветками (default и foo), двумя head и одним tip.

Команда pull в Mercurial только перемещает коммиты из удаленного репозитория в локальный, последующие слияния (или перебазирувания) задаются специальными опциями или отдельными командами.

1.2 Ссылки на коммиты и их диапазоны в Mercurial

Mercurial поддерживает развитый язык, позволяющий ссылаться на коммиты и их диапазоны. Помимо очевидных в виде начала хеша коммита и ссылки могут применяться дополнительные операции. Операция $\wedge$ указывает на предка коммита с указанным номером, т.е. $\text{tip}^\wedge$ это предок текущего головного коммита. Указывать номер предка имеет смысл для коммитов-слияний, т.к. у них имеется несколько предков – $\text{tip}^\wedge 2$ будет обозначать второго предка головного коммита, из другой ветки. Операция $\sim$ позволяет продвигаться дальше по временной линии предков. $\text{tip}^\sim 2$ будет обозначать предка предка головного коммита (т.е. через один коммит от него) и т.д. (см. рис. 2). Обратите внимание, что для операции $\sim$ нет формы без второго операнда, в отличие от $\wedge$, т.е. число указывать обязательно. Допустим, в репозитории, показанном на рисунке 2, нам нужно получить коммит, который был слит в ветку default из ветки foo предпоследним (то есть коммит G). Это можно сделать несколькими способами: получить третьего предка последнего коммита ($\text{tip}^\sim 3$), т.к. последний коммит сделан в ветку foo, получить третьего предка последнего коммита, явно указав ветку foo ($\text{foo}^\sim 3$) или получить предка второго предка предпоследнего коммита в ветке default ($\text{default}^\sim 1^\wedge 2^\sim 1$).

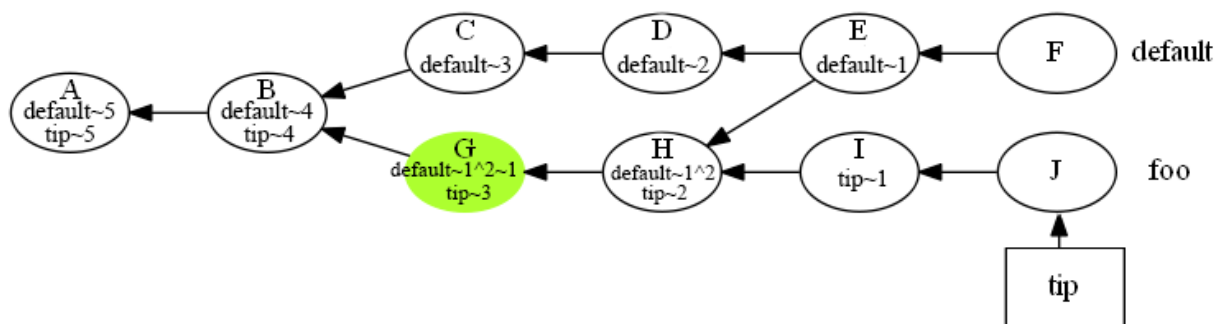


Рисунок 2. Пример комплексных ссылок на коммит

Для указания диапазонов коммитов используются операции, подобные логическим, при этом в качестве их элементов могут использоваться коммиты или множества, возвращаемые различными предикатами. Так x and y (эквивалентно $x \& y$) покажет коммиты, относящиеся к обоим множествам одновременно; x or y ($x | y$, $x + y$) – относящиеся к какому либо из них; $x - y$ покажет логическую разность (находящиеся в x , но не в y), операция $x\%y$ подобна отрицанию, но включает всех предков коммитов из множеств x и y . Восклицательный знак или not задает

отрицание, префиксная операция `::x` возвращает всех предков коммита `x`. Существует также значительное число предикатов, позволяющих получить различные множества коммитов. Так `branch(feature)` вернет все коммиты, относящиеся к данной ветке, `only(x, y)` возвращает предков `x` которые не являются предками `y` и т.д. (полный перечень представлен в <https://www.selenic.com/mercurial/hg.1.html#revsets>) Можно задавать свои предикаты, используя присваивания в специальной секции `[revsetalias]` конфигурационного файла `Mercurial`. В целом, система задания множеств коммитов в `Mercurial` является очень гибкой (реализован небольшой язык в функциональной парадигме программирования), однако мало знакома большинству пользователей.

В качестве первого примера рассмотрим вариацию команды `hg log`. Обычный `hg log -G` выглядит достаточно громоздко и даже для небольшой истории изменений может быть сложен для понимания (см. рис. 3 а). Промежуточные слияния – это слияния, после которых обе ветки продолжают развиваться отдельно (выделены серым цветом на рисунке 3 а). Как правило, они выполняются для актуализации состояния веток и не нужны для понимания истории кода. Для упрощения чтения истории можно сделать свою команду, которая будет отображать более компактный граф коммитов, без промежуточных слияний (см. рис. 3 б).

<pre>@ Änderung: 7:52fe4a8ec3cc \ Marke: tip Vorgänger: 6:7d3026216270 Vorgänger: 5:848c390645ac Nutzer: Arne Babenhauserheide <bab@draketo.de> Datum: Tue Aug 14 15:09:54 2012 +0200 Zusammenfassung: merge o Änderung: 6:7d3026216270 \ \ Vorgänger: 2:b500d0a90d40 \ \ Vorgänger: 4:8dbc55213c9f Nutzer: Arne Babenhauserheide <bab@draketo.de> Datum: Tue Aug 14 15:09:45 2012 +0200 Zusammenfassung: merged 4 o Änderung: 5:848c390645ac \ \ Vorgänger: 3:55ba56aa8299 \ \ Vorgänger: 2:b500d0a90d40 Nutzer: Arne Babenhauserheide <bab@draketo.de> Datum: Tue Aug 14 15:09:43 2012 +0200 Zusammenfassung: merged 2 +---o Änderung: 4:8dbc55213c9f \ \ Vorgänger: 3:55ba56aa8299 \ \ Vorgänger: 1:8cc66166edc9 Nutzer: Arne Babenhauserheide <bab@draketo.de> Datum: Tue Aug 14 15:09:41 2012 +0200 Zusammenfassung: merged 1 o Änderung: 3:55ba56aa8299 \ \ Vorgänger: 0:385d95ab1fea Nutzer: Arne</pre>	<pre>@ Änderung: 7:52fe4a8ec3cc \ Marke: tip Vorgänger: 6:7d3026216270 Vorgänger: 5:848c390645ac Nutzer: Arne Babenhauserheide <bab@draketo.de> Datum: Tue Aug 14 15:09:54 2012 +0200 Zusammenfassung: merge \ \ o Änderung: 3:55ba56aa8299 \ \ Vorgänger: 0:385d95ab1fea Nutzer: Arne Babenhauserheide <bab@draketo.de> Datum: Tue Aug 14 15:09:40 2012 +0200 Zusammenfassung: 4 o Änderung: 2:b500d0a90d40 \ / Vorgänger: 0:385d95ab1fea Nutzer: Arne Babenhauserheide <bab@draketo.de> Datum: Tue Aug 14 15:09:39 2012 +0200 Zusammenfassung: 3 o Änderung: 1:8cc66166edc9 \ / Nutzer: Arne Babenhauserheide <bab@draketo.de> Datum: Tue Aug 14 15:09:38 2012 +0200 Zusammenfassung: 2 o Änderung: 0:385d95ab1fea Nutzer: Arne Babenhauserheide <bab@draketo.de> Datum: Tue Aug 14 15:09:38 2012 +0200</pre>
--	---

```

Babenhauserheide <bab@draketo.de>                                Zusammenfassung: 1
| | | Datum: Tue Aug 14 15:09:40
2012 +0200
| | | Zusammenfassung: 4
| | |
| o | Änderung: 2:b500d0a90d40
| / / Vorgänger: 0:385d95ablfea
| | Nutzer: Arne
Babenhauserheide <bab@draketo.de>
| | Datum: Tue Aug 14 15:09:39
2012 +0200
| | Zusammenfassung: 3
| |
| o Änderung: 1:8cc66166edc9
| / Nutzer: Arne Babenhauserheide
<bab@draketo.de>
| Datum: Tue Aug 14 15:09:38
2012 +0200
| Zusammenfassung: 2
|
o Änderung: 0:385d95ablfea
Nutzer: Arne Babenhauserheide
<bab@draketo.de>
Datum: Tue Aug 14 15:09:38
2012 +0200
Zusammenfassung: 1

```

a)
б)
 Рисунок 3. а) пример обычного `hg log -G`, б) пример истории коммитов без отображения промежуточных слияний

Для получения графа, показанного на рисунке 3 б, можно воспользоваться системой задания множества коммитов, используя команду, показанную на рисунке 4.

```
hg log -Gr "(all() - merge()) or head()"
```

Рисунок 4. Пример `hg log` без отображения промежуточных слияний

В данной команде содержится три предиката. Предикат `all()` возвращает все изменения (коммиты) (тоже, что `"0:tip"`), предикат `merge()` возвращает набор коммитов, содержащий только слияния и предикат `head()` возвращающий головную ревизию. Нам необходимо из множества всех ревизий вычесть слияния, но добавить головные независимо от того, являются ли они слияниями, что и достигается операциями разности и объединения множеств в указанном выражении.

В качестве второго примера рассмотрим использование `hg diff` в совокупности с системой задания диапазона коммитов. Периодически возникает потребность найти коммит, в котором были внесены изменения. Известен примерный период, в котором был сделан коммит: например с марта по апрель 2017 года. Также известен программист, который его сделал и слово, упомянутое в списке изменений. Чтобы найти точный набор изменений в заданном промежутке дат с заданным словом, можно воспользоваться командой, показанной на рисунке 5.

```
$ hg log -r "date('>05/01/17') and
date('<06/01/17') and user(vasiliy) and
desc('refactoring')"
```


Рисунок 5. Пример поиска коммитов с заданными параметрами

Результатом работы команды будут все изменения, в которых дата была между 1 мая 2017 года и 1 июня 2017 года, сделанная пользователем, чье имя содержит строку «vasiliy» и где описание коммита включает в себя слово «refactoring».

Часто в проектах присутствует несколько веток, живущих достаточно долго. Эти ветки периодически сливаются друг с другом. При разработке новой функциональности важно иметь в feature-ветке актуальные коммиты из основной ветки проекта, чтобы гарантировать, что исправление ошибок также распространяется и на feature-ветку. Иногда при слияниях появляются конфликты, и, если изменения особенно запутанные, эти конфликты может быть довольно сложно разрешить. Манипуляция наборами изменений (revsets) может помочь в решении и этой проблемы. Есть несколько наборов изменений, содержащих полезную информацию: последний раз, когда основная ветка разработки была слита с текущей веткой; самый последний общий предок двух наборов изменений и т.д. Например, допустим, в середине слияния желательно увидеть изменения, которые были сделаны в конкретном файле между последним слиянием (общий предок для двух изменяемых веток) и набором изменений, которые мы пытаемся в настоящее время слить (второй родитель). Можно узнать все различия между общим предком двух веток и вторым родителем из текущего рабочего каталога (см.рис.105).

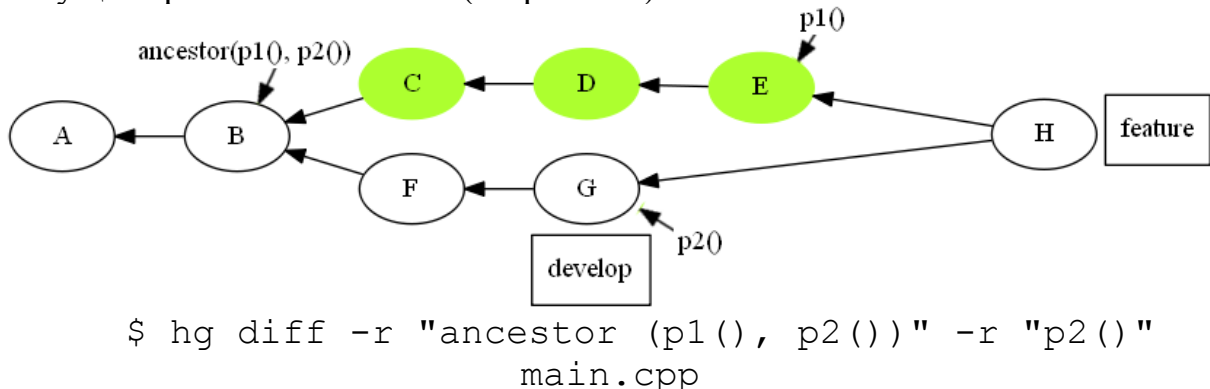
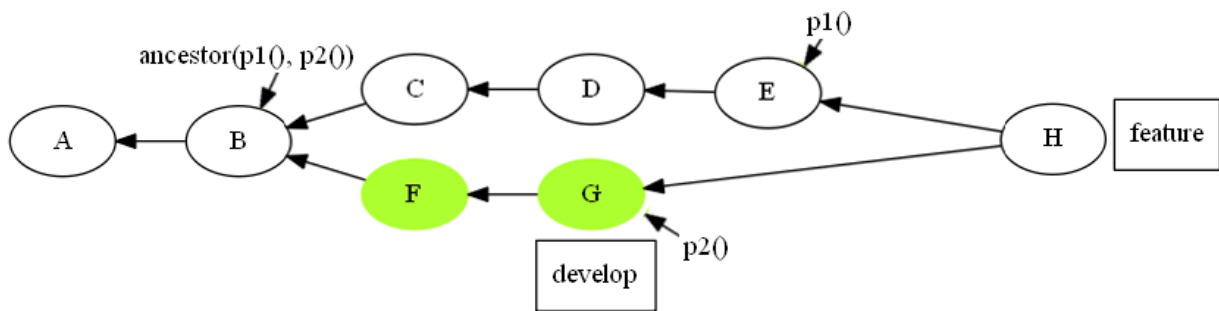


Рисунок 105. Просмотр списка изменений заданного файла между последним слиянием и набором изменений, которые подлежат слиянию

Результатом работы команды на рисунке 105 будет разница между общим предком `p1()` (локального родителя) и `p2()` (другим родителем / набором изменений, который мы объединяем) и `p2()`. Это покажет все изменения, произошедшие в другой ветке разработки, в файле `main.cpp`.

Чтобы увидеть все изменения, произошедшие в feature-ветке в конкретном файле, нужно просто заменить в команде, показанной на рисунке 2, вторую ревизию на `p1()`, как показано на рисунке 7.



```
$ hg diff -r "ancestor (p1 (), p2 ())" -r "p1 ()"
main.cpp
```

Рисунок 7. Просмотр всех изменений заданного файла в feature-ветке

1.3. Особенности работы в системе Git

Git в применении обычных команд сильнее отличается от классических централизованных систем контроля версий, чем Mercurial. Это связано, в частности, с тем, что Линус Торвалдс при разработке Git пользовался принципом «от противного» относительно популярной централизованной системы CVS, делая при каждом удобном случае наоборот. Хотя это обеспечило Git много сильных и эффективных сторон, это также привело к заменам удобного и привычного интерфейса пользователя там, где этого совсем не требовалось или к спорным решениям. Поэтому работа некоторых команд git (add, commit, reset, checkout) выглядит несколько эзотеричной и требует понимания особенностей его устройства.

В дополнение к двум элементам обычной системы контроля версий: файлам и репозиторию – Git вводит еще один – индекс. Индекс содержит изменения, которые будут включены в следующий коммит. В типовом рабочем цикле вы не просто делаете коммит, а сначала заносите желаемые изменения в индекс (командами git add и git remove), можете перепроверить себя, посмотрев состояние индекса (команда git status) и, наконец, переместить изменения из индекса в коммит (git commit). Обратите внимание, что это изменяет привычную семантику команды add – если в большинстве систем контроля версий она выполняется только при добавлении новых файлов под контроль версий, то в Git она заносит добавленные или измененные файлы в индекс, и должна выполняться всякий раз после изменения файла перед коммитом. Наличие индекса позволяет вам подготовить изменения, включенные в коммит, отдельно от содержимого рабочего каталога (так что легко закоммитить его часть), перепроверить себя перед коммитом, а также содержит интерфейс выбора ханков, включаемых в коммит (git add --patch).

Допустим, у нас есть репозиторий, в котором не было никаких изменений. Выполним команду git status, чтобы убедиться, что изменений не было (см. рис. 8).

```
$ git status
```

```
# On branch master
nothing to commit, working directory clean
```

Рисунок 8. Статус репозитория без изменений

Добавим в проект файл `main.cpp`. Так как такого файла раньше в проекте не было, `git status` покажет не отслеживаемый файл (см. рис. 9).

```
$ git status
# On branch master
# Untracked files:
#   (use "git add <file>..." to include in what will be
committed)
#
#   main.cpp
# nothing added to commit but untracked files present (use
"git add" to track)
```

Рисунок 9. Статус репозитория при добавлении нового файла в рабочий каталог

Тот факт, что файл `main.cpp` является не отслеживаемым, по сути означает, что Git видит файл, которого не было в предыдущем снимке состояния (коммите). Git не станет добавлять файл в коммит, пока этого явно не указать. Для того, чтобы добавить файл под версионный контроль, необходимо выполнить `git add` (см. рис. 10).

```
$ git add main.cpp
```

Рисунок 10. Добавление файла в индекс

Теперь при помощи команды `git status` можно убедиться, что `main.cpp` теперь отслеживаемый и индексируемый (см. рис. 11).

```
$ git status
# On branch master
# Changes to be committed:
#   (use "git reset HEAD <file>..." to unstage)
#
#   new file:   main.cpp
```

Рисунок 11. Статус репозитория при добавленном в индекс файле

Если внести изменения в файл `functions.cpp`, который уже есть в индексе, а затем выполнить `git status`, то команда покажет, что файл `functions.cpp` был изменен (см. рис. 12).

```
$ git status
# On branch master
# Changes to be committed:
#   (use "git reset HEAD <file>..." to unstage)
#
#   new file:   main.cpp
```

```

#
# Changes not staged for commit:
#   (use "git add <file>..." to update what will be
committed)
#
#    modified:   functions.cpp

```

Рисунок 12. Статус репозитория при добавленном файле в индекс и измененном индексируемом файле

Файл `functions.cpp` находится в секции “Changes not staged for commit”, что означает, что этот файл был изменён, но пока не проиндексирован. Чтобы проиндексировать его, необходимо выполнить команду `git add` (см. рис. 13).

```

$ git add functions.cpp
$ git status
# On branch master
# Changes to be committed:
#   (use "git reset HEAD <file>..." to unstage)
#
#    new file:   main.cpp
#    modified:   functions.cpp

```

Рисунок 13. Добавление изменения в индекс

Внесем изменение в `functions.cpp`, и проверим его статус (см. рис. 14).

```

$ git status
# On branch master
# Changes to be committed:
#   (use "git reset HEAD <file>..." to unstage)
#
#    new file:   main.cpp
#    modified:   functions.cpp
#
# Changes not staged for commit:
#   (use "git add <file>..." to update what will be
committed)
#
#    modified:   functions.cpp

```

Рисунок 14. Статус репозитория при добавленном в индекс новом и изменённом файлах и измененном индексируемом файле

Теперь файл `functions.cpp` отображается в блоке проиндексированных и не проиндексированных файлов. Это связано с тем, что Git индексирует файл в точности в том состоянии, в котором он находился, когда была выполнена команда `git add`. Если выполнить коммит, то в него попадут те изменения, которые попали в индекс при последнем выполнении команды `git add`, а не те, которые актуальны на данный момент.

Чтобы удалить файл из Git’a, нужно сначала удалить его из индекса, а затем сделать коммит. Это можно сделать при помощи команды `git rm`, которая также удаляет файл из рабочего каталога. Если просто удалить

файл из рабочего каталога, он будет показан в секции “Changes not staged for commit” вывода команды `git status` (см. рис. 15).

```
$ rm functions.cpp
$ git status
# On branch master
#
# Changes not staged for commit:
#   (use "git add/rm <file>..." to update what will be
committed)
#
#       deleted:    functions.cpp
```

Рисунок 15. Статус репозитория при удаленном из рабочего каталога файле

После выполнения команды `git rm`, удаление файла попадет в индекс (см. рис. 16).

```
$ git rm functions.cpp
rm ' functions.cpp '
$ git status
# On branch master
#
# Changes to be committed:
#   (use "git reset HEAD <file>..." to unstage)
#
#       deleted:    functions.cpp
```

Рисунок 16. Удаление файла из индекса с удалением его из рабочего каталога

Если необходимо удалить файл из индекса, при этом оставив его в рабочем каталоге, то необходимо выполнить `git rm` с опцией `-cached` (см. рис. 17).

```
$ git rm --cached functions.cpp
```

Рисунок 17. Удаление файла из индекса с оставлением его в рабочем каталоге

Преимущества такого подхода не очень велики, т.к. GUI клиенты большинства систем контроля версий имеют схожие функции (и более удобны за счет графического интерфейса). К недостаткам же следует отнести большее количество команд в типовой операции коммита и дополнительные шансы на неполный коммит за счет отсутствия измененных файлов. В обычной системе контроля версий забываются, как правило, только новые файлы, т.к. все измененные предлагаются к коммиту по умолчанию. На практике, в условиях ограниченного времени, программисты часто делают коммиты в спешке, не уделяя им достаточно внимания (будучи сосредоточенными на проблемах программного кода), от чего дополнительная команда не очень помогает, а временами и мешает. Кроме того, такая осторожность в создании коммитов несколько нелогична именно для Git, в котором коммиты легко редактируются и даже полностью удаляются. Впрочем, любители традиционного интерфейса, могут использовать команду `commit` с опцией `-a` для автоматического

включения в коммит всех измененных версионизируемых файлов без предварительных добавлений их в индекс (см. рис. 18).

```
$ git status
# On branch master
# Changes to be committed:
#   (use "git reset HEAD <file>..." to unstage)
#
#   new file:   main.cpp
#
# Changes not staged for commit:
#   (use "git add <file>..." to update what will be committed)
#   (use "git checkout - <file>..." to discard changes in working
directory)
#
#   modified:   functions.php
#
$ git commit -a -m 'добавлена новая функция'
[master 5c25980] 7777
2 files changed, 45 insertions(+), 1 deletion(-)
create mode 100644 main.cpp
$ git status
# On branch master
nothing to commit (working directory clean)
```

Рисунок 18. Коммит без предварительных изменений в индекс

Другой особенностью Git является отказ от стандартной команды `update`, обеспечивавшей переключение между ревизиями, в пользу команды `checkout`, которая используется как для переключения между ветками, так и ревизиями. Для того, чтобы правильно понять работу этой команды необходимо разобраться в системе ссылок (`refs`) в Git.

С точки зрения Git, ссылки это указатели на коммиты, которые используются (в частности) для реализации веток (указатель переставляется при коммите на вершину ветки) и меток (указатель остается на месте). Любой коммит надежно существует только до тех пор, пока он «доступен» по какой-либо ссылке, т.е. является предком коммита, на которую указывает эта ссылка – доступность идет назад по линии времени. Недоступные коммиты могут быть уничтожены сборщиком мусора, хотя это обычно происходит далеко не сразу (в пределах нескольких месяцев). Сослаться на недоступный коммит можно только по его хешу, который еще нужно как-то записать или узнать.

По умолчанию, во вновь созданном репозитории создается указатель ветки `master` (аналог ветки `default` в Mercurial) и специальный системный указатель `HEAD` (не путать с понятием `head` в Mercurial, их смысл различный), который определяет точку, в которую будет сделан очередной коммит – т.е. при выполнении команды `git commit` коммит, на который указывает `HEAD`, становится предком нового. В нормальном состоянии `HEAD` указывает не напрямую на коммиты, а на ветку, выбранную текущей (так что переставляется вместе с ней). Команда `checkout`

перемещает HEAD (и обновляет содержимое рабочего каталога) в указанное место – обычно на новую ветку, но при указании конкретного коммита перемещает HEAD на него – т.е. может быть использована вместо традиционной в других системах команды update. Такое состояние называется «отсоединенным HEAD» т.к. HEAD не указывает ни на одну из веток (см. рис. 19-21).

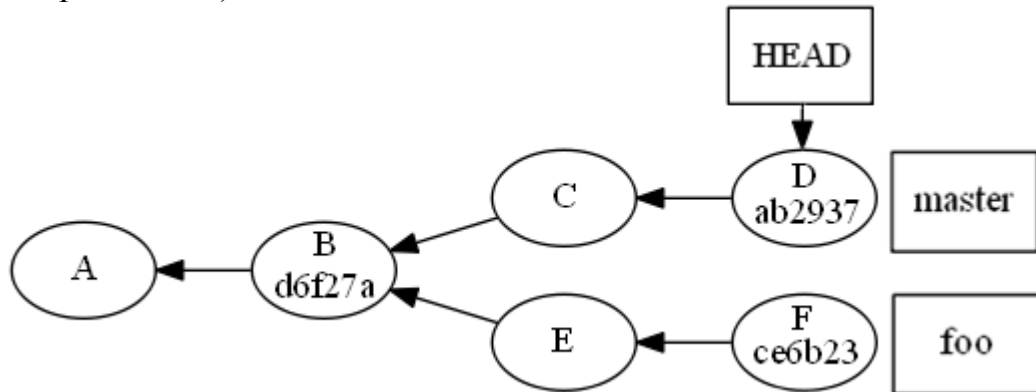


Рисунок 19. Пример состояния репозитория с присоединенным HEAD

```
$ git checkout d6f27a
Note: checking out 'd6f27a'.
```

You are in 'detached HEAD' state. You can look around, make experimental changes and commit them, and you can discard any commits you make in this state without impacting any branches by performing another checkout.

If you want to create a new branch to retain commits you create, you may do so (now or later) by using -b with the checkout command again. Example:

```
git checkout -b new_branch_name
```

HEAD is now at d6f27a... First Commit

Рисунок 20. Пример использования git checkout

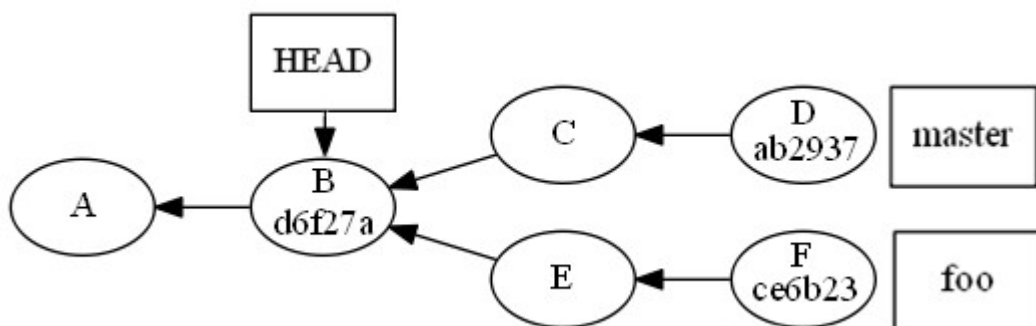


Рисунок 21. Пример состояния репозитория с отсоединенным HEAD

Если сделать в этом состоянии коммит, то он произведет анонимную ветку, которая может быть уничтожена сборщиком мусора, если позже не создать на ее окончание метку или ветку. Такое действие обычно является ошибкой, поэтому перед фиксацией коммита проверьте, что вы не видите сообщение о «Detached HEAD».

В отличие от Mercurial, для простого вытягивания коммитов из удаленного репозитория в Git используется команда `fetch`. Команда `pull` вытягивает ветку и сливает ее с текущей (точнее с местом, на который указывает HEAD), так что при ее применении следует быть осторожным чтобы не произвести нежелательных слияний.

1.4. Ссылки на коммиты и их диапазоны в Git

При составлении ссылок на предков текущих коммитов операции `^` и `~` работают в Git так же, как и в Mercurial (см. раздел 4.2.2).

Для указания диапазона коммитов используется синтаксис двух или трех точек, которые имеют разный смысл. При применении двух точек будут показаны все коммиты, достижимые из правой ссылки и не достижимые из левой. Т.е. при наличии двух веток `master` и `feature` `master..feature` будет обозначать все коммиты в `feature` которых нет в `master` (а `feature..master` – наоборот). Это также удобно при просмотре линейной истории с ограничением по глубине: `master~3..master` покажет три последних коммита ветки `master`, а `master~5..master~2` покажет коммиты, отстоящие от вершины `master` на 2, 3 и 4 коммита соответственно. В общем случае вы можете составить выражение из коммитов, перечисляя те из них, предков которых надо добавить просто так, а коммиты (ветки), предков которых надо исключить – предваряя `^`. Так `master..feature` эквивалентно `^master feature`. В случае, если в нашем репозитории есть 3 ветки (`master`, `bugfixes`, `feature`), 2 из которых были слиты (`master` и `bugfixes`), и мы хотим посмотреть, каких же коммитов не хватает в нашей `feature` ветке из ветки `master`, за исключением коммита слияния, нам нужно выполнить команду `git log feature..master~1` (см. рис. 22).

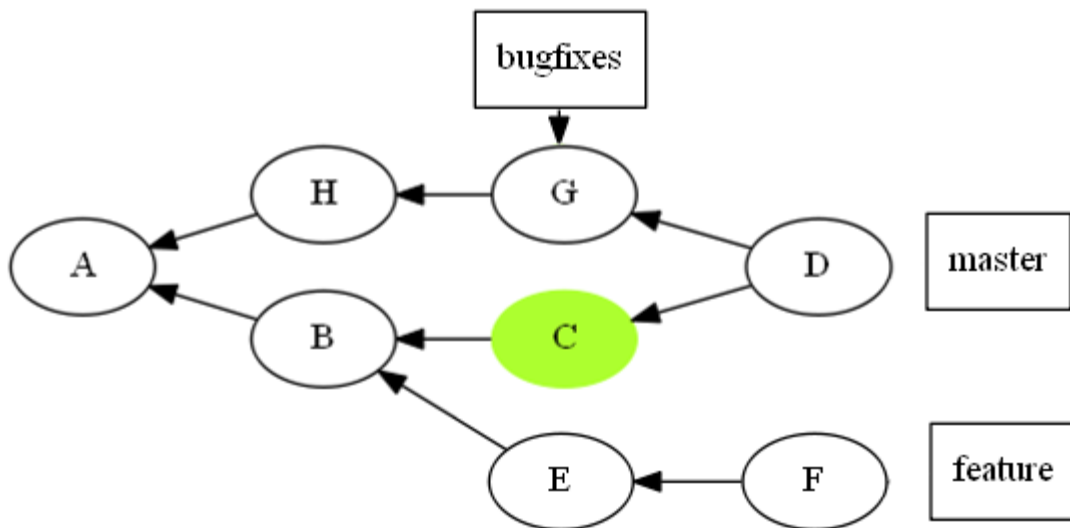


Рисунок 22. Зеленым выделены коммиты, возвращаемые выражением `feature..master~1`

Запись из трех точек покажет коммиты, которые достижимы по одной из ссылок, но не по обеим сразу – т.е. `master...feature` покажет и `master..feature`, и `feature..master`. В общем случае вы можете вместо диапазона перечислить несколько ссылок на ветки, перед некоторыми из которых может быть указан знак ^, обозначающий отрицание (т.е. берутся коммиты, недостижимые из них). Допустим, мы хотим получить для предыдущего примера все коммиты, которые есть в ветке `feature` и `master`, за исключением мержа `bugfixes` в `master` и коммитов из `bugfixes`, для этого воспользуемся командой `git log (master~1...feature)...^bugfixes` (см. рис. 23).

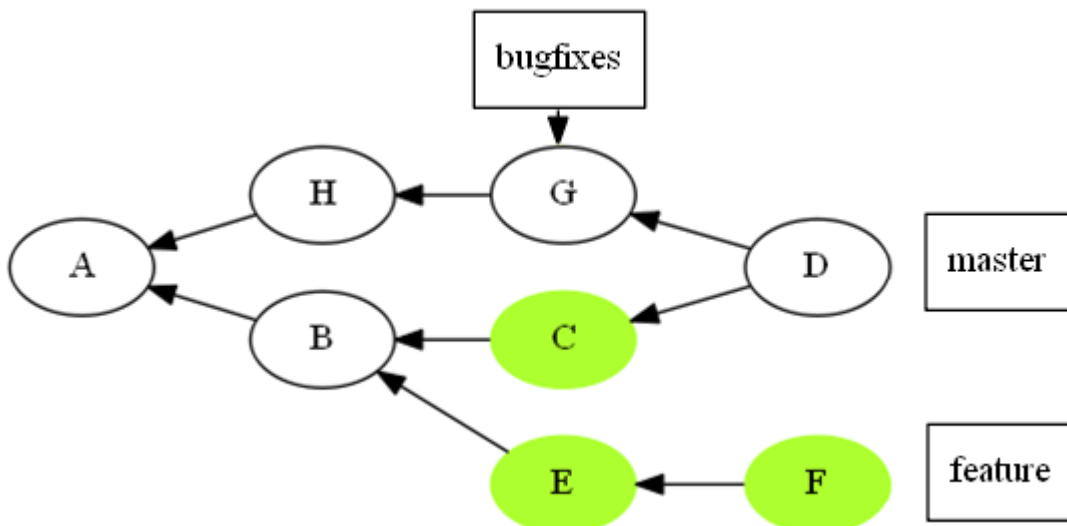


Рисунок 23. Зеленым выделены коммиты, возвращаемые выражением `(master~1...feature)...^bugfixes`